

project.) in the project's log. The output shows that we're on the main branch ❷ and that two files have changed ❸.

When we check the status now, we can see that we're on the main branch, and we have a clean working tree ❹. This is the message you should see each time you commit a working state of your project. If you get a different message, read it carefully; it's likely you forgot to add a file before making a commit.

## Checking the Log

Git keeps a log of all commits made to the project. Let's check the log:

---

```
git_practice$ git log
commit cea13ddc51b885d05a410201a54faf20e0d2e246 (HEAD -> main)
Author: eric <eric@example.com>
Date:   Mon Jun 6 19:37:26 2022 -0800
```

```
    Started project.
git_practice$
```

---

Each time you make a commit, Git generates a unique, 40-character reference ID. It records who made the commit, when it was made, and the message recorded. You won't always need all of this information, so Git provides an option to print a simpler version of the log entries:

---

```
git_practice$ git log --pretty=oneline
cea13ddc51b885d05a410201a54faf20e0d2e246 (HEAD -> main) Started project.
git_practice$
```

---

The `--pretty=oneline` flag provides the two most important pieces of information: the reference ID of the commit and the message recorded for the commit.

## The Second Commit

To see the real power of version control, we need to make a change to the project and commit that change. Here we'll just add another line to *hello\_git.py*:

---

```
hello_git.py print("Hello Git world!")
              print("Hello everyone.")
```

---

When we check the status of the project, we'll see that Git has noticed the file that changed:

---

```
git_practice$ git status
❶ On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
```